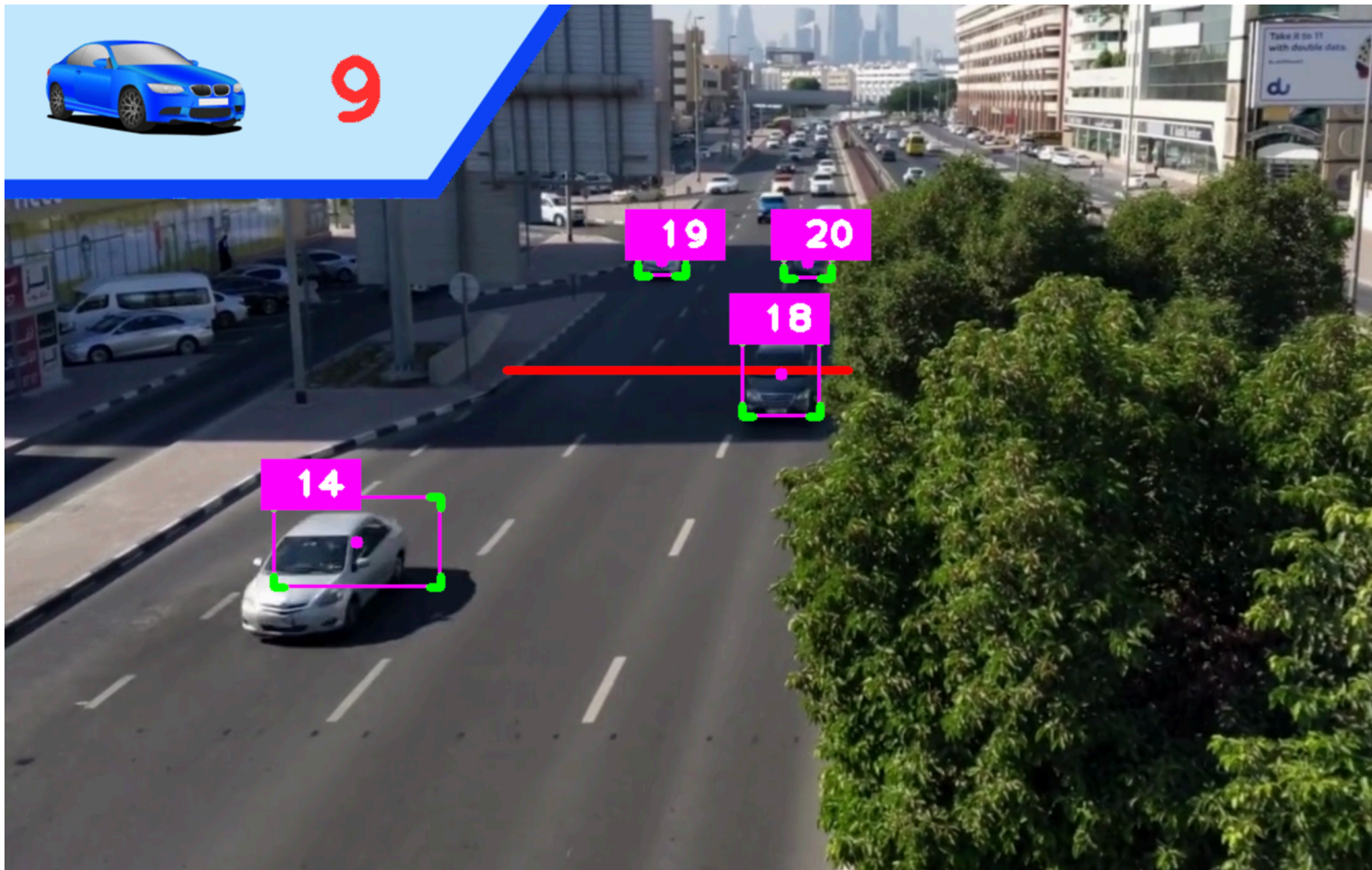


YOLO-Car-Counter

차량이 line에 도착하면 count



YOLO-Car-Counter

차량이 line에 도착하면 count



```
mask = cv2.imread("mask.png")
limits = [400, 297, 673, 297]
totalCount = []
```

```
cv2.line(img, (limits[0], limits[1]), (limits[2], limits[3]), (0, 0, 255), 5)
for result in resultsTracker:
    x1, y1, x2, y2, id = result
    x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)
    print(result)
    w, h = x2 - x1, y2 - y1
    cvzone.cornerRect(img, (x1, y1, w, h), l=9, rt=2, colorR=(255, 0, 255))
    cvzone.putTextRect(img, f' {int(id)}', (max(0, x1), max(35, y1)),
                      scale=2, thickness=3, offset=10)
```



```
cx, cy = x1 + w // 2, y1 + h // 2
cv2.circle(img, (cx, cy), 5, (255, 0, 255), cv2.FILLED)
```

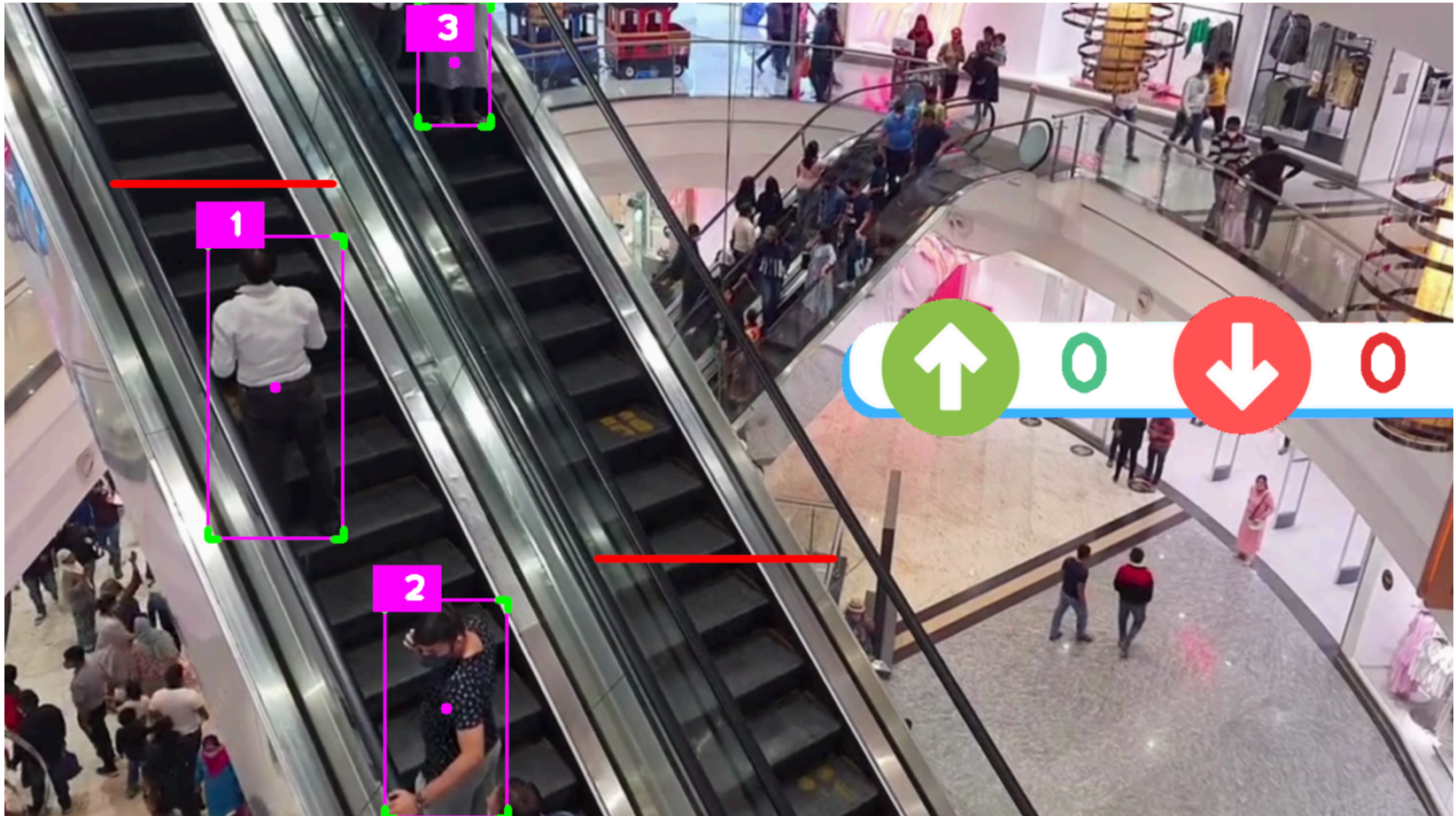


```
if limits[0] < cx < limits[2] and limits[1] - 15 < cy < limits[1] + 15:
    if totalCount.count(id) == 0:
        totalCount.append(id)
        cv2.line(img, (limits[0], limits[1]), (limits[2], limits[3]), (0, 255, 0), 5)
```

```
cv2.putText(img, str(len(totalCount)), (255, 100), cv2.FONT_HERSHEY_PLAIN, 5, (50, 50, 255), 8)
```

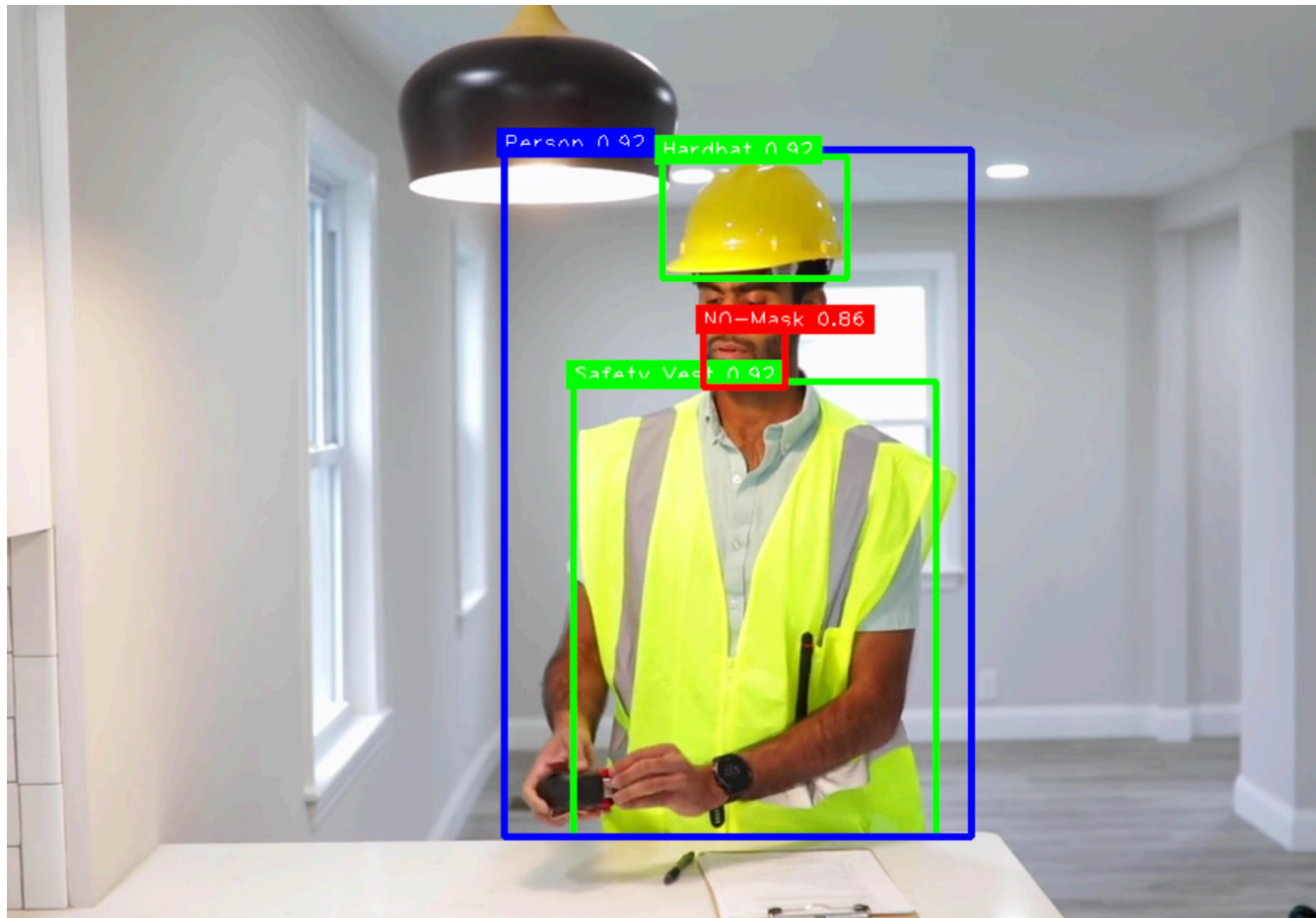
YOLO-People Counter

YOLO-People-Counter



YOLO-PPE Detection

YOLO-People-Counter



YOLO-PPE Detection

```
from ultralytics import YOLO
import cv2
import cvzone
import math

cap = cv2.VideoCapture("../Videos/ppe-1.mp4") # For Video

model = YOLO("ppe.pt")

classNames = [
    "Hardhat",
    "Mask",
    "NO-Hardhat",
    "NO-Mask",
    "NO-Safety Vest",
    "Person",
    "Safety Cone",
    "Safety Vest",
    "machinery",
    "vehicle",
]
```

YOLO-PPE Detection

```
myColor = (0, 0, 255)
while True:
    success, img = cap.read()
    results = model(img, stream=True)
    for r in results:
        boxes = r.boxes
        for box in boxes:
            # Bounding Box
            x1, y1, x2, y2 = box.xyxy[0]
            x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

            w, h = x2 - x1, y2 - y1

            # Confidence
            conf = math.ceil((box.conf[0] * 100)) / 100
            # Class Name
            cls = int(box.cls[0])
            currentClass = classNames[cls]
            print(currentClass)
```


YOLO-PPE Detection

```
print(currentClass)
if conf > 0.5:
    if (
        currentClass == "NO-Hardhat"
        or currentClass == "NO-Safety Vest"
        or currentClass == "NO-Mask"
    ):
        myColor = (0, 0, 255)
    elif (
        currentClass == "Hardhat"
        or currentClass == "Safety Vest"
        or currentClass == "Mask"
    ):
        myColor = (0, 255, 0)
    else:
        myColor = (255, 0, 0)

    cvzone.putTextRect(
        img,
        f"{classNames[cls]} {conf}",
        (max(0, x1), max(35, y1)),
        scale=1,
        thickness=1,
        colorB=myColor,
        colorT=(255, 255, 255),
        colorR=myColor,
        offset=5,
    )
    cv2.rectangle(img, (x1, y1), (x2, y2), myColor, 3)
```

```
cv2.imshow("Image", img)
cv2.waitKey(1)
```

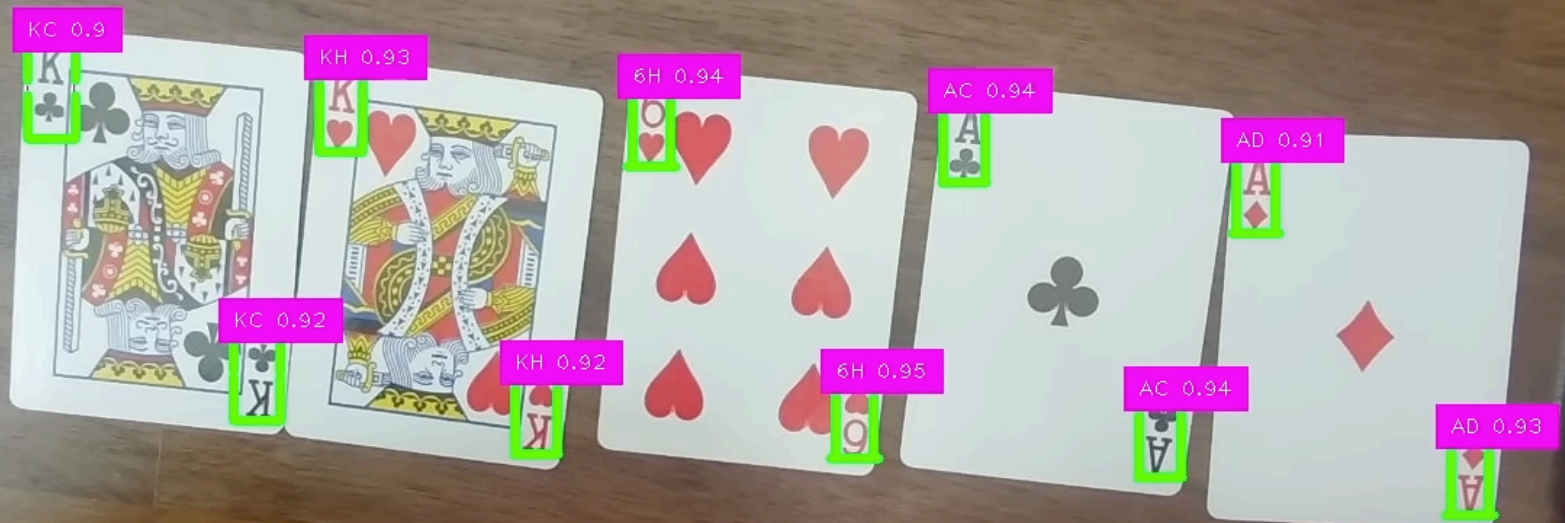
YOLO-PokerHand Detection

YOLO-PokerHand Detection

Royal Flush					
Straight Flush					
4 of a Kind					
Full House					
Flush					
Straight					
3 of a Kind					
2 Pairs					
1 Pair					
High Card					

YOLO-PokerHand Detection

Your Hand: Two Pair



PokerHandFunction.py

```
def findPokerHand(hand):
    ranks = []
    suits = []
    possibleRanks = []

    for card in hand:
        if len(card) == 2:
            rank = card[0]
            suit = card[1]
        else:
            rank = card[0:2]
            suit = card[2]
        if rank == "A":
            rank = 14
        elif rank == "K":
            rank = 13
        elif rank == "Q":
            rank = 12
        elif rank == "J":
            rank = 11
        ranks.append(int(rank))
        suits.append(suit)

    sortedRanks = sorted(ranks)
```

PokerHandFunction.py

```
sortedRanks = sorted(ranks)

# Royal Flush and Straight Flush and Flush
if suits.count(suits[0]) == 5: # Check for Flush
    if 14 in sortedRanks and 13 in sortedRanks and 12 in sortedRanks and 11 in sortedRanks \
        and 10 in sortedRanks:
        possibleRanks.append(10)
    elif all(sortedRanks[i] == sortedRanks[i - 1] + 1 for i in range(1, len(sortedRanks))):
        possibleRanks.append(9)
    else:
        possibleRanks.append(6) # -- Flush

# Straight
# 10 11 12 13 14
# 11 == 10 + 1
if all(sortedRanks[i] == sortedRanks[i - 1] + 1 for i in range(1, len(sortedRanks))):
    possibleRanks.append(5)

handUniqueVals = list(set(sortedRanks))
```


PokerHandFunction.py

```
# Four of a kind and Full House
# 3 3 3 3 5 -- set --- 3 5 --- unique values = 2 --- Four of a kind
# 3 3 3 5 5 -- set -- 3 5 ---- unique values = 2 --- Full house
if len(handUniqueVals) == 2:
    for val in handUniqueVals:
        if sortedRanks.count(val) == 4: # --- Four of a kind
            possibleRanks.append(8)
        if sortedRanks.count(val) == 3: # --- Full house
            possibleRanks.append(7)

# Three of a Kind and Pair
# 5 5 5 6 7 -- set -- 5 6 7 --- unique values = 3 -- three of a kind
# 8 8 7 7 2 -- set -- 8 7 2 --- unique values = 3 -- two pair
if len(handUniqueVals) == 3:
    for val in handUniqueVals:
        if sortedRanks.count(val) == 3: # -- three of a kind
            possibleRanks.append(4)
        if sortedRanks.count(val) == 2: # -- two pair
            possibleRanks.append(3)
```

PokerHandFunction.py

```
# Pair
# 5 5 3 6 7 -- set -- 5 3 6 7 - unique values = 4 -- Pair
if len(handUniqueVals) == 4:
    possibleRanks.append(2)

if not possibleRanks:
    possibleRanks.append(1)
# print(possibleRanks)
pokerHandRanks = {10: "Royal Flush", 9: "Straight Flush", 8: "Four of a Kind",
                  7: "Full House", 6: "Flush",
                  5: "Straight", 4: "Three of a Kind", 3: "Two Pair", 2: "Pair",
                  1: "High Card"}
output = pokerHandRanks[max(possibleRanks)]
print(hand, output)
return output

if __name__ == "__main__":
    findPokerHand(["KH", "AH", "QH", "JH", "10H"]) # Royal Flush
    findPokerHand(["QC", "JC", "10C", "9C", "8C"]) # Straight Flush
    findPokerHand(["5C", "5S", "5H", "5D", "QH"]) # Four of a Kind
    findPokerHand(["2H", "2D", "2S", "10H", "10C"]) # Full House
    findPokerHand(["2D", "KD", "7D", "6D", "5D"]) # Flush
    findPokerHand(["JC", "10H", "9C", "8C", "7D"]) # Straight
    findPokerHand(["10H", "10C", "10D", "2D", "5S"]) # Three of a Kind
    findPokerHand(["KD", "KH", "5C", "5S", "6D"]) # Two Pair
    findPokerHand(["2D", "2S", "9C", "KD", "10C"]) # Pair
    findPokerHand(["KD", "5H", "2D", "10C", "JH"]) # High Card
```

YOLO-PokerHand Detection

```
from ultralytics import YOLO
import cv2
import cvzone
import math
import PokerHandFunction

cap = cv2.VideoCapture(cv2.CAP_DSHOW + 0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

model = YOLO("playingCards.pt")
classNames = [
    "10C",
    "10D",
    "10H",
    "10S",

    "QH",
    "QS",
]
```

YOLO-PokerHand Detection

```
while True:
    success, img = cap.read()
    results = model(img, stream=True)
    hand = []
    for r in results:
        boxes = r.boxes
        for box in boxes:
            # Bounding Box
            x1, y1, x2, y2 = box.xyxy[0]
            x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

            w, h = x2 - x1, y2 - y1
            cvzone.cornerRect(img, (x1, y1, w, h))
            # Confidence
            conf = math.ceil((box.conf[0] * 100)) / 100
            # Class Name
            cls = int(box.cls[0])

            cvzone.putTextRect(
                img,
                f"{classNames[cls]} {conf}",
                (max(0, x1), max(35, y1)),
                scale=1,
                thickness=1,
            )

            if conf > 0.5:
                hand.append(classNames[cls])
```

YOLO-PokerHand Detection

```
print(hand)
hand = list(set(hand))
print(hand)
if len(hand) == 5:
    results = PokerHandFunction.findPokerHand(hand)
    print(results)
    cvzone.putTextRect(
        img, f"Your Hand: {results}", (300, 75), scale=3, thickness=5
    )

cv2.imshow("Image", img)
cv2.waitKey(1)
```